



DevOps

Kubernetes Volumes

Kubernetes Volumes

- In Kubernetes, there are several types of volumes that can be used to provide persistent storage to your applications:
 - **EmptyDir:** This type of volume is created within a pod's container and has a lifetime that is tied to the pod. The data stored in an EmptyDir volume is deleted when the pod is deleted or restarted.
 - **HostPath:** This type of volume mounts a file or directory from the host node's file system into the pod. This can be useful for accessing host-specific files or for sharing data between containers running on the same node.
 - **ConfigMap:** This type of volume allows you to store configuration data in key-value pairs, which can be accessed by your application as files in a directory within the pod's container.

Kubernetes Volumes

- **Secret:** Similar to ConfigMap, Secret volume allows you to store sensitive information like passwords, tokens, or certificates in key-value pairs, which can be accessed as files in a directory within the pod's container.
- **NFS:** This type of PV uses the NFS protocol to access storage resources that are hosted on a remote NFS server.
- **PersistentVolumeClaim (PVC):** This type of volume allows you to claim a persistent volume from a storage system that has been provisioned by a cluster administrator. This volume can then be used by your application, and the data stored within it persists even if the pod is deleted.

Kubernetes Volumes

- AWS EBS: This type of PV uses Amazon Web Services (AWS) Elastic Block Store (EBS) to provision and manage storage resources on AWS.
- Azure Disk: This type of PV uses Microsoft Azure Disk to provision and manage storage resources on Azure.
- GCE Persistent Disk: This type of PV uses Google Compute Engine (GCE) Persistent Disk to provision and manage storage resources on Google Cloud Platform (GCP).

emptyDir

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-app
  template:
    metadata:
      labels:
        app: nginx-app
    spec:
      containers:
        - name: nginx-cont
          image: nginx
          ports:
            - containerPort: 80
          volumeMounts:
            - name: my-vol
              mountPath: /data
      volumes:
        - name: my-vol
          emptyDir: {}
```

hostPath

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-app
  template:
    metadata:
      labels:
        app: nginx-app
    spec:
      containers:
        - name: nginx-cont
          image: nginx
          ports:
            - containerPort: 80
          volumeMounts:
            - name: my-vol
              mountPath: /data
      volumes:
        - name: my-vol
          hostPath:
            path: /opt/data
```

ConfigMap

```
# No ConfigMap
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres-app
  template:
    metadata:
      labels:
        app: postgres-app
    spec:
      containers:
        - name: postgres-cont
          image: postgres
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_PASSWORD
              value: "123"
```

ConfigMap - Example 1

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres-app
  template:
    metadata:
      labels:
        app: postgres-app
    spec:
      containers:
        - name: postgres-cont
          image: postgres
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_PASSWORD
              valueFrom:
                configMapKeyRef:
                  name: postgres-configmap
                  key: pass
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-configmap
data:
  pass: "123"
```

```
$ kubectl get configmap
$ kubectl describe configmap postgres-configmap
```


ConfigMap - Example 2

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres-app
  template:
    metadata:
      labels:
        app: postgres-app
    spec:
      containers:
        - name: postgres-cont
          image: postgres
          ports:
            - containerPort: 5432
          envFrom:
            - configMapRef:
                name: postgres-configmap
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-configmap
data:
  POSTGRES_PASSWORD: "123"
```

ConfigMap - Example 3

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: index-configmap
data:
  index.html: |-
    <!DOCTYPE html>
    <html>
    <head>
      <title>My Simple HTML Page</title>
    </head>
    <body>
      <h1>Welcome to DARA</h1>
      <p>This is a simple HTML page.</p>
      <ul>
        <li>Item 1</li>
        <li>Item 2</li>
        <li>Item 3</li>
      </ul>
    </body>
    </html>
```

ConfigMap

- Example 3

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx-container
          image: nginx:latest
          ports:
            - containerPort: 80
          volumeMounts:
            - name: config-volume
              mountPath: /usr/share/nginx
      volumes:
        - name: config-volume
          configMap:
            name: index-configmap
            items:
              - key: index.html
                path: html/index.html
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      nodePort: 31111
```

Secret

```
# No Secret
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres-app
  template:
    metadata:
      labels:
        app: postgres-app
    spec:
      containers:
        - name: postgres-cont
          image: postgres
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_PASSWORD
              value: "123"
```

Secret

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres-deploy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres-app
  template:
    metadata:
      labels:
        app: postgres-app
    spec:
      containers:
        - name: postgres-cont
          image: postgres
          ports:
            - containerPort: 5432
          envFrom:
            - secretRef:
                name: postgres-secret
```

```
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
type: Opaque
data:
  POSTGRES_PASSWORD: MTIzCg==
# echo -n "123" | base64
# echo -n "MTIzCg==" | base64 --decode
```

```
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
type: Opaque
stringData:
  POSTGRES_PASSWORD: "123"
```

Secret with Private Registry

- `$ vim config.json`

```
{
  "auths": {
    "registry.e-crops.co": {
      "username": "dara",
      "password": "123",
      "email": "dara@gmail.com",
      "auth": "ZGFyYToxMjM="
    }
  }
}
```

=> `echo -n "dara:123" | base64`

- `$ base64 -w 0 config.json`
- Copy the TOKEN

```
leangsengk90@node1:~/my-kube$ base64 -w 0 config.json
ewogICJhdXRocyI6IHsKICAgICJyZWdpc3RyeS5lLWNyb3BzLmNvIjogewogICAgIC
AidXNlcm5hbWUiOiAiZGFyYSIsCiAgICAgICJwYXNzd29yZCI6ICxMjMiLAogICAg
ICAiZW1haWwiOiAiZGFyYUBnbWVpbC5jb20iLAogICAgICAiYXV0aCI6ICJaR0Z5WV
RveE1qTT0iCiAgICB9CiAgfQp9Cgo=leangsengk90@node1:~/my-kube$
```

Secret with Private Registry

- `$ vim secret.yml`

```
apiVersion: v1
kind: Secret
metadata:
  name: registry-secret
type: kubernetes.io/dockerconfigjson
data:
  .dockerconfigjson: >-
    ewogICJhdXRocyI6IHsKICAgICJyZWdpc3RyeS51LWNyb3BzLmNvIjogewogICAgICAi
dXNlcm5hbWUiOiAiZGFyYSIsCiAgICAgICJwYXNzd29yZCI6ICIxMjMiLAogICAgICAiZW1h
aWwiOiAiZGFyYUBnbWFpbC5jb20iLAogICAgICAiYXV0aCI6ICJaR0Z5WVRveE1qTT0iCiAg
ICB9CiAgfQp9Cgo=
```

Secret with Private Registry

- `$ vim deploy.yml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-nginx-deploy
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
```

```
selector:
  matchLabels:
    tier: my-nginx
template:
  metadata:
    labels:
      tier: my-nginx
  spec:
    imagePullSecrets:
      - name: registry-secret
    containers:
      - name: my-nginx-cont
        image: registry.e-crops.co/my-nginx:1.1.1
        ports:
          - containerPort: 80
```


Network File System(NFS)

- **On Master Server**

- `$ sudo apt update`
- `$ sudo apt install nfs-kernel-server -y`
- `$ sudo mkdir /opt/nfs/data -p`
- `$ sudo chown nobody:nogroup /opt/nfs/data`
- `$ sudo chmod 777 -R /opt/nfs/data`
- `$ exit`
- `$ sudo nano /etc/exports`
 - `/opt/nfs/data      own-self-ip(rw, sync, no_subtree_check) client-ip2(rw, sync, no_subtree_check)`
 - `/opt/nfs/data      *(rw, sync, no_subtree_check) # All IPs`
- `sudo systemctl restart nfs-kernel-server`

Network File System(NFS)

- **On Slave Server**

- `$ sudo apt update`
- `$ sudo apt install nfs-common -y`
- `$ sudo mkdir /opt/nfs/data -p`
- `$ sudo chown nobody:nogroup /opt/nfs/data`
- `$ sudo chmod 777 -R /opt/nfs/data`
- `$ exit`
- `$ sudo mount master-server-ip:/opt/nfs/data /opt/nfs/data`
- Create file to test in `/opt/nfs/data` and mount with volumes

Network File System(NFS)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: register-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: register-app
  template:
    metadata:
      labels:
        app: register-app
    spec:
      nodeSelector:
        group: workers
      containers:
        - name: register-cont
          image: xeng/spring-register:7
          ports:
            - containerPort: 8080
          volumeMounts:
            - name: register-vol
              mountPath: /src/main/resources/images
      volumes:
        - name: register-vol
          nfs:
            server: 10.148.0.12
            path: /opt/nfs/data
```

```
$ kubectl label node node_name group=workers
$ kubectl get node --show-labels
```

Network File System(NFS)

```
apiVersion: v1
kind: Service
metadata:
  name: register-svc
spec:
  type: NodePort
  selector:
    app: register-app
  ports:
    - port: 8080
      nodePort: 32222
```

PV and PVC

- PV and PVC are two different types of storage in Kubernetes that can be used to manage persistent data in a containerized environment.
- PV stands for "Persistent Volume" and is an abstract representation of a storage volume in Kubernetes. A PV is created by an administrator and represents a piece of storage in the cluster. It can be used by multiple pods, and it's not tied to any specific node. A PV can be dynamically provisioned or statically defined by an administrator.
- On the other hand, PVC stands for "Persistent Volume Claim" and represents a request for storage by a pod. PVCs are created by developers and are used to request a specific amount of storage with a specific access mode. A PVC can be used by a single pod, and it's tied to that pod's lifecycle.

PV and PVC

- There are three types of access modes that can be used to define how a Persistent Volume (PV) can be accessed by pods:
 1. **ReadWriteOnce** (RWO): The volume can be mounted as read-write by a single node in the cluster. This access mode is useful for scenarios where a pod needs to write to the volume and other pods should not be able to read or write to it simultaneously.
 2. **ReadOnlyMany** (ROX): The volume can be mounted as read-only by many nodes in the cluster. This access mode is useful for scenarios where multiple pods need to read from the same volume but should not be able to modify it.
 3. **ReadWriteMany** (RWX): The volume can be mounted as read-write by many nodes in the cluster. This access mode is useful for scenarios where multiple pods need to read from and write to the same volume simultaneously.

PV and PVC

- There are three types of reclaim policies that can be used for a Persistent Volume (PV):
 1. **Retain:** When a PV is bound to a Persistent Volume Claim (PVC), the PV is preserved even after the PVC is deleted. The data on the PV is retained and can be manually reclaimed or deleted by an administrator. This policy is useful when the data on the PV is critical or needs to be retained for compliance purposes.
 2. **Delete:** When a PVC is deleted, the corresponding PV is also deleted and the storage resources are released. This is the default reclaim policy. This policy is useful for scenarios where the data on the PV is not critical and can be easily regenerated or does not need to be retained.
 3. **Recycle:** When a PVC is deleted, the corresponding PV is formatted and the storage resources are released. This policy is useful for scenarios where the data on the PV is not critical and the PV can be reused by a new PVC. However, this policy is not commonly used in modern Kubernetes clusters and has been deprecated in Kubernetes 1.17.

PV and PVC: Static Provisioning

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: my-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteMany
  storageClassName: nfs-sharing
  nfs:
    server: 10.148.0.12
    path: /opt/nfs/data
```

```
# Patch PV from Released to Available
$ kubectl patch pv my-pv -p '{"spec":{"claimRef": null}}'
```


PV and PVC: Static Provisioning

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Gi
  storageClassName: nfs-sharing
```

PV and PVC: Static Provisioning

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: register-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: register-app
  template:
    metadata:
      labels:
        app: register-app
    spec:
      containers:
        - name: register-cont
          image: xeng/spring-register:7
          ports:
            - containerPort: 8080
          volumeMounts:
            - name: register-vol
              mountPath: /src/main/resources/images
      volumes:
        - name: register-vol
          persistentVolumeClaim:
            claimName: my-pvc
```

PV and PVC: Static Provisioning

```
apiVersion: v1
kind: Service
metadata:
  name: register-svc
spec:
  type: NodePort
  selector:
    app: register-app
  ports:
    - port: 8080
      nodePort: 32222
```

PV and PVC: Dynamic Provisioning

- **Resource:**

- <https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner>
- <https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner/tree/master/deploy>

```
$ helm repo add nfs-subdir-external-provisioner
https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner/

$ helm install nfs-subdir-external-provisioner
nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
  --set nfs.server=10.148.0.2 \
  --set nfs.path=/opt/nfs/data

$ kubectl get pod -n nfs-prov
$ kubectl get sc
$ kubectl get pv, pvc
$ kubectl delete pvc # Delete pvc and it will delete pv but it stored in nfs dir.
```

PV and PVC: Dynamic Provisioning

```
$ vim pvc.yml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  storageClassName: nfs-client
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 2Gi
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nfs-pod
spec:
  containers:
    - name: app-container
      image: nginx
      volumeMounts:
        - mountPath: /mnt/data
          name: nfs-volume
  volumes:
    - name: nfs-volume
      persistentVolumeClaim:
        claimName: myclaim
```

A large, faint watermark of the ISTAD logo is centered in the background. It is a circular emblem with a blue outer ring containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and Khmer text. Inside the ring is a blue circle with a white atomic symbol. The word 'ISTAD' is written in red across the bottom of the inner circle. Binary code '01010100 01000001 01000000' is visible along the top inner edge of the ring.

Thank you

Begin with the theory, Start with the practical